

Cluster Apache Spark con Ansible

Iván Domínguez y Lucas Miranda

Este documento detalla el procedimiento que hemos seguido para iniciar el cluster de Apache Spark con Ansible usando Docker en lugar de máquinas virtuales (resultaba más cómodo hacerlo conociendo esta herramienta).

Entorno: macOS (Apple Silicon) Docker Ansible Apache Spark 3.5.1.

1 Introducción y arquitectura

El objetivo es desplegar un cluster de Apache Spark compuesto por un nodo maestro y dos workers, gestionados íntegramente con Ansible.

Para no depender de máquinas virtuales reales, se utilizan tres contenedores Docker que Ansible trata exactamente igual que VMs físicas.

La arquitectura del cluster es:

- spark-master: nodo maestro Spark (expone UI en puerto 8080)
- spark-worker-1: primer nodo worker
- spark-worker-2: segundo nodo worker

2 Requisitos previos

Instalar las dependencias necesarias:

```
# Instalar Ansible y librerías Docker para Python
pip3 install ansible molecule molecule-docker docker
```

```
# Verificar instalaciones
ansible --version
molecule --version
```

Docker debe estar corriendo. En Mac sin Docker Desktop se puede usar Colima:

```
brew install colima docker
colima start
```

3 Estructura del proyecto

Crear la siguiente estructura de carpetas:

```
spark-cluster/  
  inventory/  
    hosts.ini  
  roles/  
    spark/  
      tasks/  
        main.yml  
      templates/  
        spark-env.sh.  
  playbook.yml  
  docker-compose.yml
```

4 Levantar los nodos con Docker Compose

El archivo *docker-compose.yml* define los tres contenedores Ubuntu que simulan las VMs. La imagen base instala Python3 al arrancar, necesario para que Ansible pueda operar sobre ellos.

docker-compose.yml

```
services:  
  spark-master:  
    image: ubuntu:22.04  
    container_name: spark-master  
    hostname: spark-master  
    command: /bin/bash -c "apt-get update -qq && apt-get install -y -qq python3 && tail -f /dev/null"  
    ports:  
      - "8080:8080"  
    networks:  
      - spark-net  
  
  spark-worker-1:  
    image: ubuntu:22.04  
    container_name: spark-worker-1  
    hostname: spark-worker-1  
    command: /bin/bash -c "apt-get update -qq && apt-get install -y -qq python3 && tail -f /dev/null"  
    networks:  
      - spark-net  
  
  spark-worker-2:  
    image: ubuntu:22.04  
    container_name: spark-worker-2
```

```

hostname: spark-worker-2
command: /bin/bash -c "apt-get update -qq && apt-get install -y -qq python3 && tail -f /
networks:
    - spark-net

networks:
    spark-net:
        driver: bridge

Arrancar los contenedores:

docker-compose up -d

```

5 Inventario de Ansible

El inventario define los hosts que Ansible gestionará. Se usa `ansible_connection=docker` para que Ansible entre directamente en los contenedores sin necesidad de SSH ni claves.

inventory/hosts.ini

```

[master]
spark-master ansible_host=spark-master ansible_user=root ansible_connection=docker

[workers]
spark-worker-1 ansible_host=spark-worker-1 ansible_user=root ansible_connection=docker
spark-worker-2 ansible_host=spark-worker-2 ansible_user=root ansible_connection=docker

[spark:children]
master
workers

```

Verificar que Ansible ve los tres nodos:

```
ansible all -i inventory/hosts.ini -m ping
```

Resultado esperado:

```

spark-master | SUCCESS => {
    "ping": "pong"
}
spark-worker-1 | SUCCESS => {
    "ping": "pong"
}
spark-worker-2 | SUCCESS => {
    "ping": "pong"
}

```

6 Playbook de instalación de Spark

El playbook realiza tres plays en orden: instala Java y Spark en todos los nodos, arranca el master y finalmente conecta los workers al master.

playbook.yml

```
- name: Instalar y configurar Apache Spark
  hosts: spark
  become: yes
  vars:
    spark_version: "3.5.1"
    hadoop_version: "3"
    spark_home: "/opt/spark"
    ansible_python_interpreter: /usr/bin/python3.10
  tasks:
    - name: Instalar dependencias
      apt:
        name:
          - openjdk-11-jdk
          - wget
          - curl
        state: present
        update_cache: yes

    - name: Descargar Spark
      get_url:
        url: "https://archive.apache.org/dist/spark/spark-{{ spark_version }}/spark-{{ spark_version }}-bin-hadoop{{ hadoop_version }}.tgz"
        dest: "/tmp/spark.tgz"

    - name: Extraer Spark
      unarchive:
        src: "/tmp/spark.tgz"
        dest: "/opt/"
        remote_src: yes

    - name: Crear enlace simbolico
      file:
        src: "/opt/spark-{{ spark_version }}-bin-hadoop{{ hadoop_version }}"
        dest: "{{ spark_home }}"
        state: link

    - name: Configurar variables de entorno
      lineinfile:
        path: /etc/environment
        line: "{{ item }}"
```

```

    loop:
      - 'SPARK_HOME=/opt/spark'
      - 'PATH=$PATH:/opt/spark/bin'

- name: Arrancar Spark Master
  hosts: master
  become: yes
  vars:
    ansible_python_interpreter: /usr/bin/python3.10
  tasks:
    - name: Iniciar master
      shell: /opt/spark/sbin/start-master.sh
      environment:
        JAVA_HOME: /usr/lib/jvm/java-11-openjdk-arm64

- name: Arrancar Spark Workers
  hosts: workers
  become: yes
  vars:
    ansible_python_interpreter: /usr/bin/python3.10
  tasks:
    - name: Iniciar worker
      shell: /opt/spark/sbin/start-worker.sh spark://spark-master:7077
      environment:
        JAVA_HOME: /usr/lib/jvm/java-11-openjdk-arm64

```

Ejecutar el playbook:

```
ansible-playbook -i inventory/hosts.ini playbook.yml
```

7 Verificación del cluster

Comprobar que los procesos de Spark están corriendo en cada nodo:

```

docker exec spark-master bash -c "ps aux | grep spark"
docker exec spark-worker-1 bash -c "ps aux | grep spark"
docker exec spark-worker-2 bash -c "ps aux | grep spark"

```

Acceder a la UI web del Spark Master abriendo en el navegador:

```
http://localhost:8080
```

En la interfaz web se pueden ver el master activo y los dos workers conectados, confirmando que el cluster está operativo.

El resultado es exitoso y podemos comprobar que nuestro cluster de Spark está corriendo sin mayores problemas:

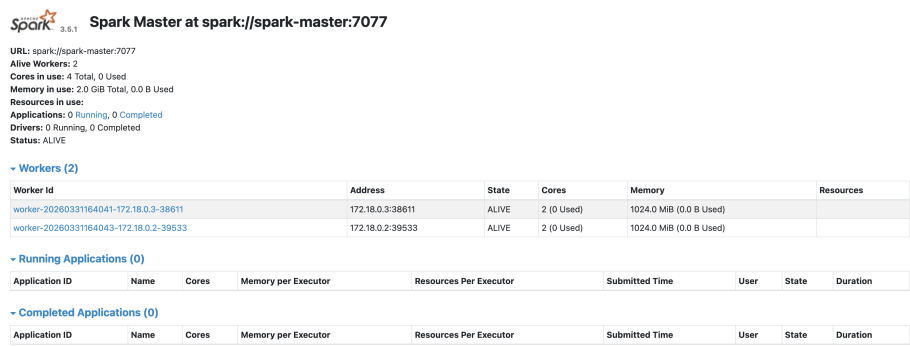


Figure 1: Servicio del nodo maestro del cluster de Spark creado